

Answer Key — Operator Overloading & the Anatomy of a Copy

Instructor reference for the review worksheet

array[0] Fill in the Blank

1. `*this`
2. `const`
3. `*this` (the calling object)
4. `one / two`
5. `encapsulation`
6. `copy` constructor
7. `ints`
8. `Fraction&` (a reference to `*this`)

array[1] Find the Error

Case A `a.num / a.den` are `private`; this free `operator+` has no access. Fix: add `friend Fraction operator+(const Fraction&, const Fraction&);` inside the class — the function signature doesn't change.
DOES NOT COMPILE

Case B `operator+` is missing `const`, so it can't be invoked on `a` or `b`, which are `const Fraction&`. Fix: `Fraction operator+(const Fraction& rhs) const { ... }`.
DOES NOT COMPILE

Case C Comparing raw `num / den` fails for unreduced-but-equal fractions — $1/2$ and $2/4$ are the same value but different fields. Fix: cross-multiply — `return num * rhs.den == rhs.num * den;`
COMPILES, WRONG ANSWER

Case D `a + 2` resolves through the *member* `operator+` found on `a`'s type, implicitly converting `2` to `Fraction` for the parameter. `2 + a` looks for a member `operator+` on `int` — which doesn't exist — and there's no non-member `operator+(int, const Fraction&)` to fall back on. Fix: a non-member (or `friend`) `operator+` makes both orders resolve.
LINE 13 DOES NOT COMPILE

Case E The compiler-generated copy constructor copies the `data` pointer's *address*, not the array it points to. `b1` and `b2` now both “own” the same heap block — a textbook shallow copy, setting up a double free / dangling pointer once destructors (even implicit ones) run.
COMPILES, UNDEFINED BEHAVIOR

array[2] Short Answer — model answers

1. *Why const:* without it, `operator+` isn't callable on a `const Fraction` or through a `const Fraction&`. Example failing line: `void f(const Fraction& a, const Fraction& b) { a + b; }` — won't compile if `operator+` isn't `const`.
2. *Member vs. non-member:* `cout << obj`'s left operand is an `ostream` — you can't add members to a type you don't own, so the overload must be free (or `friend`). `a + b`'s left operand is your own `Fraction`, which you're free to add a member to, so a member overload is the natural, simpler choice.
3. *Buffer's crash:* both `Buffer` objects end up pointing at the same heap array; whichever is destroyed first (or whichever writes) leaves the other holding a dangling pointer, and any code that frees both frees the same block twice. Term: **shallow copy**.
4. *Why Fraction is safe:* its members are plain value types (`int`) with no ownership semantics, so a member-wise copy is already a complete, independent copy. A raw owning pointer (like `Buffer::data`) is what forces a hand-written copy constructor, destructor, and `operator=` together — the “Rule of Three.”